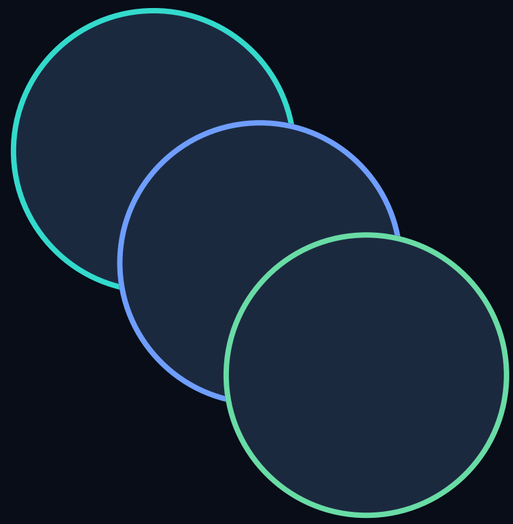
SOFTWARE VERIFICATION •

Durable Workflow Engine

How real-world systems keep multi-step processes moving when services fail



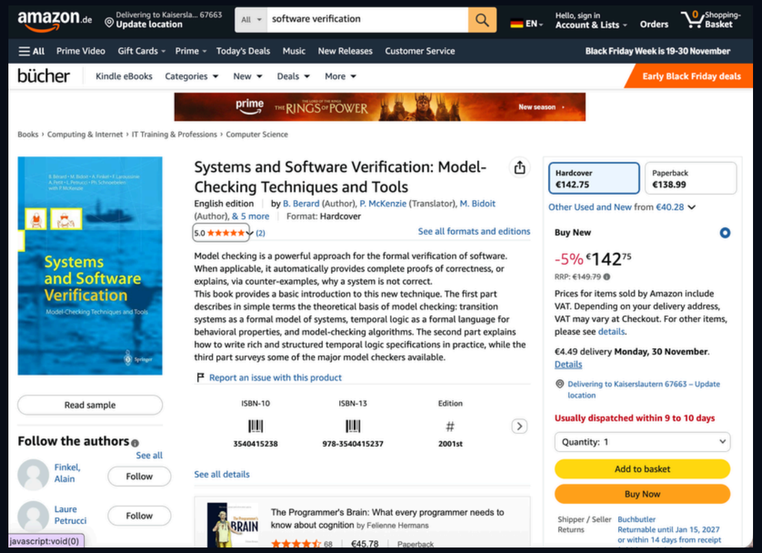
Durable state · Safe retries · Recovery · Compensation

TypeScript · Node.js · SQLite (WAL) · Fastify

SOFTWARE VERIFICATION •

What happens after you click “Buy Now”?

Designing a durable workflow engine for processes that must survive failures, retries and restarts.

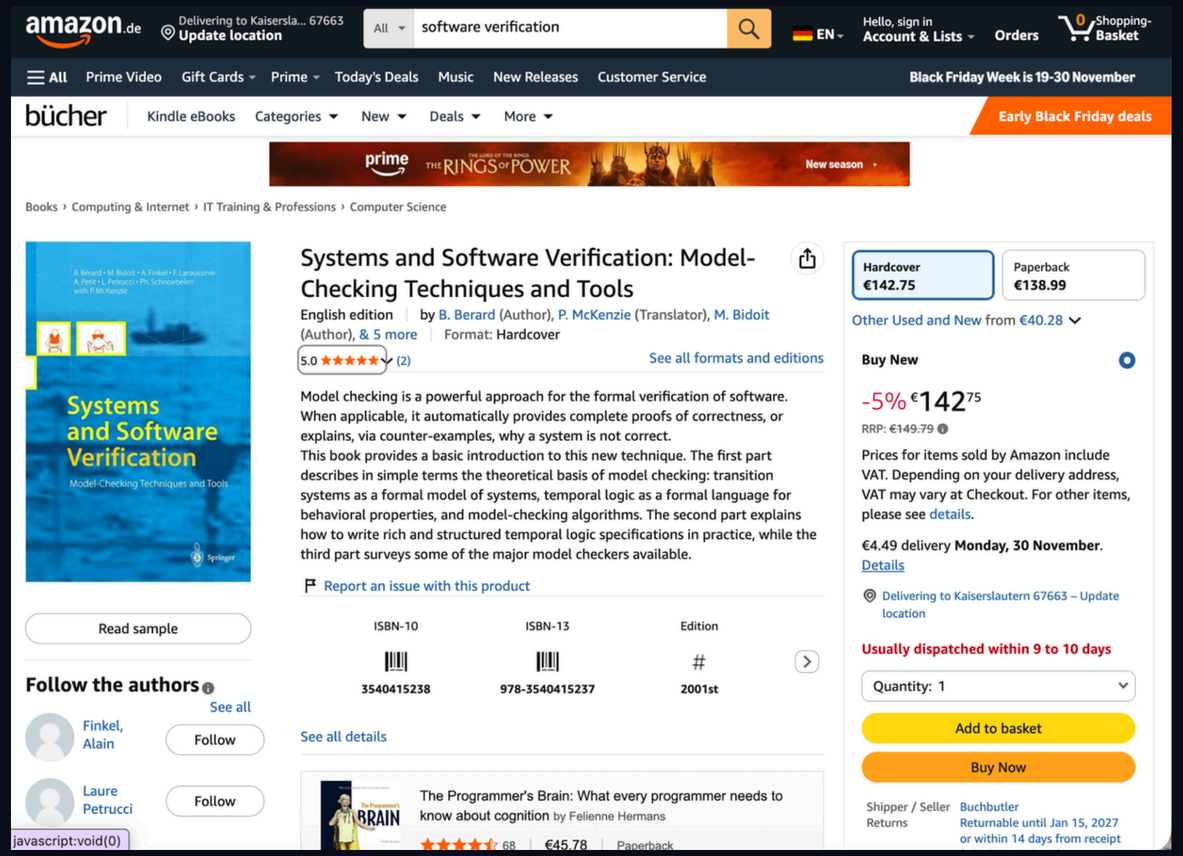


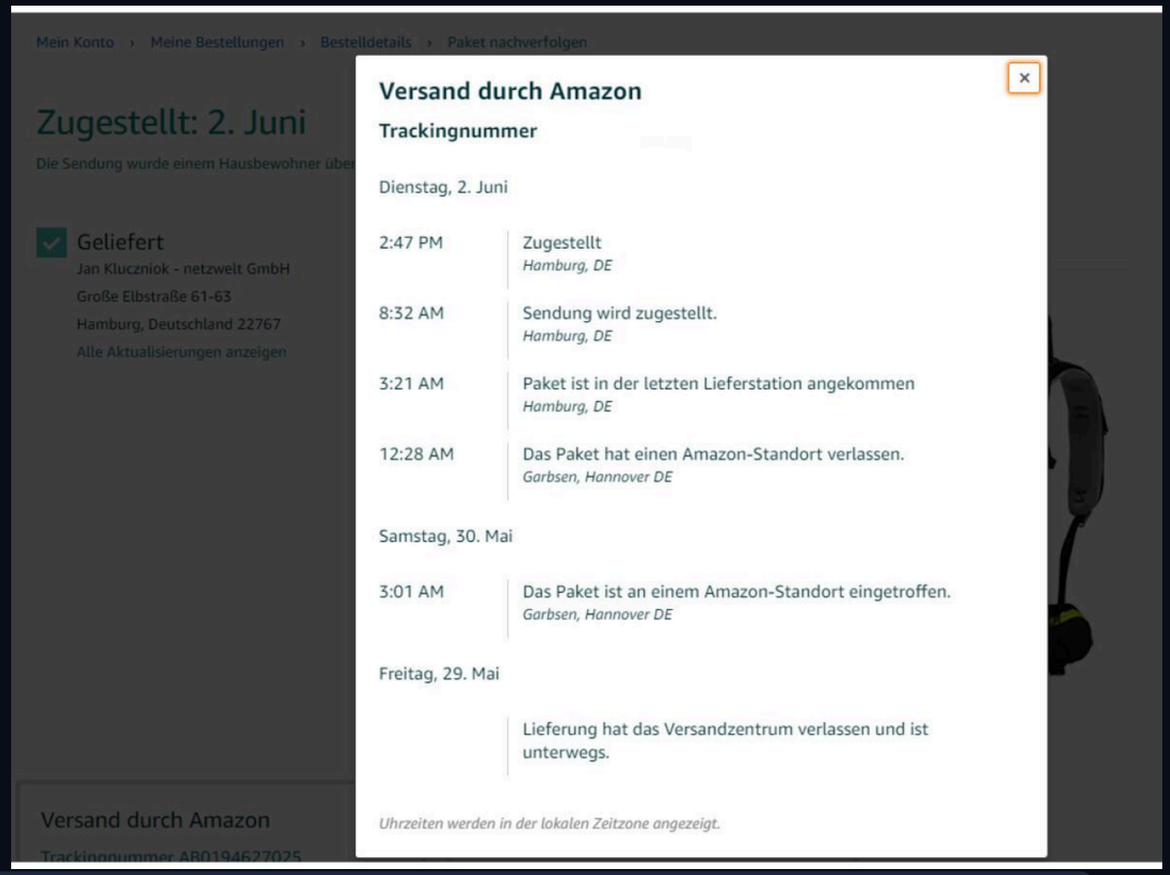
DURABLE STATE · SAFE RETRIES · RECOVERY · COMPENSATION

A lightweight system inspired by durable workflow platforms

A simple purchase hides a multi-step process

The Amazon screenshots give us the customer-facing view.

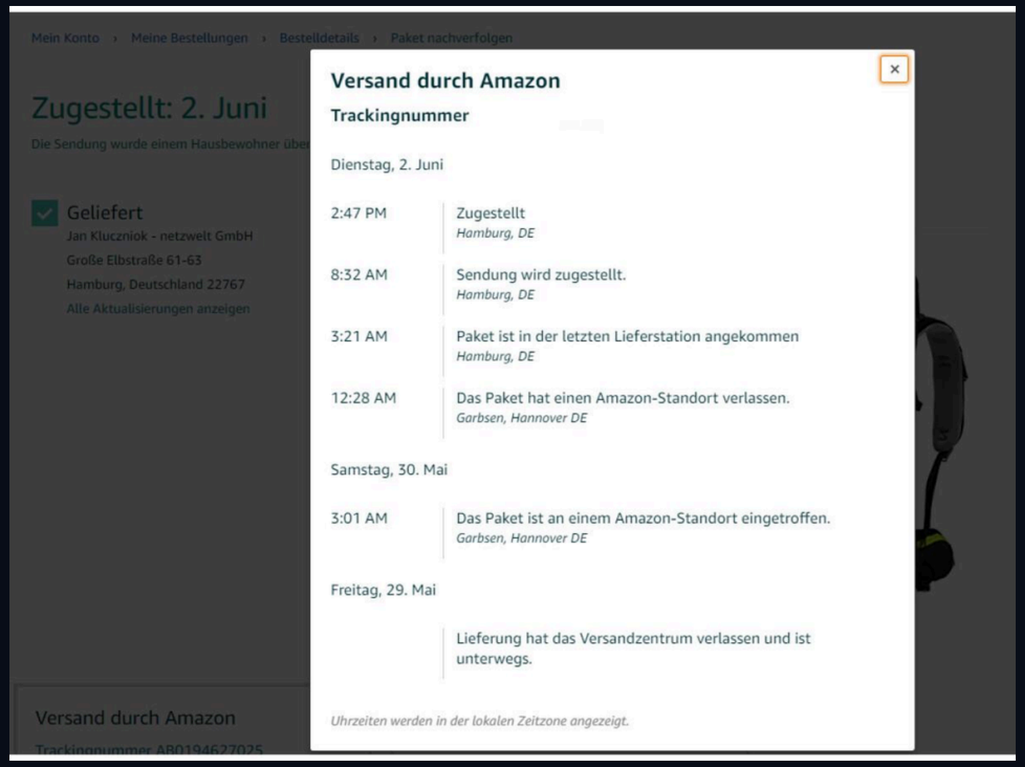




The customer sees a product page and tracking updates. The backend must coordinate many operations.

The visible delivery timeline is a workflow

The screenshot shows shipment events.

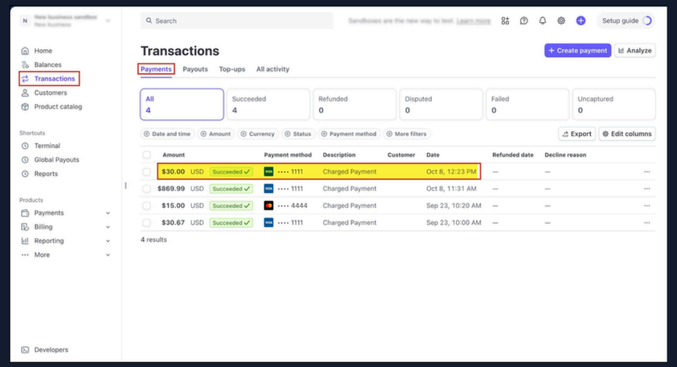




Each stage can depend on another service. A failure between stages must not lose the workflow's progress.

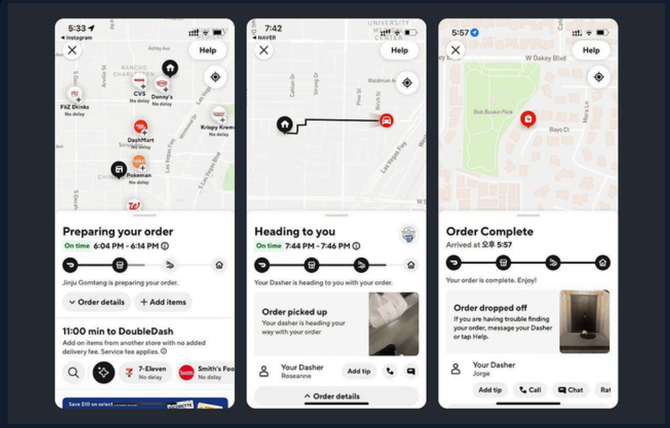
Different products. The same coordination problem.

These familiar interfaces show how multi-step operations appear in everyday technology.



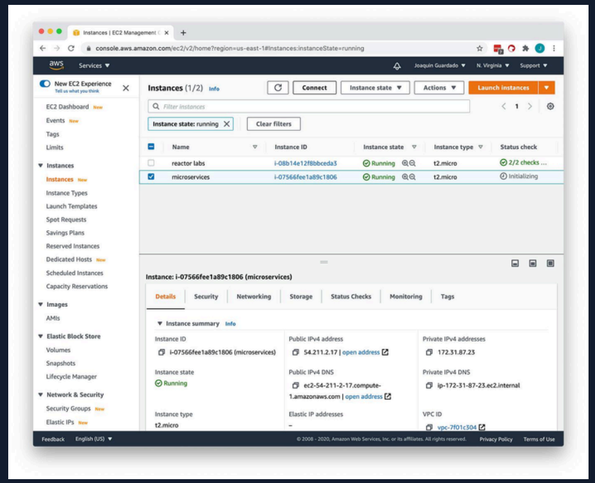
Payments

A transaction moves through statuses such as succeeded, refunded or failed.



Food delivery

Restaurant preparation, courier pickup and delivery progress must stay coordinated.

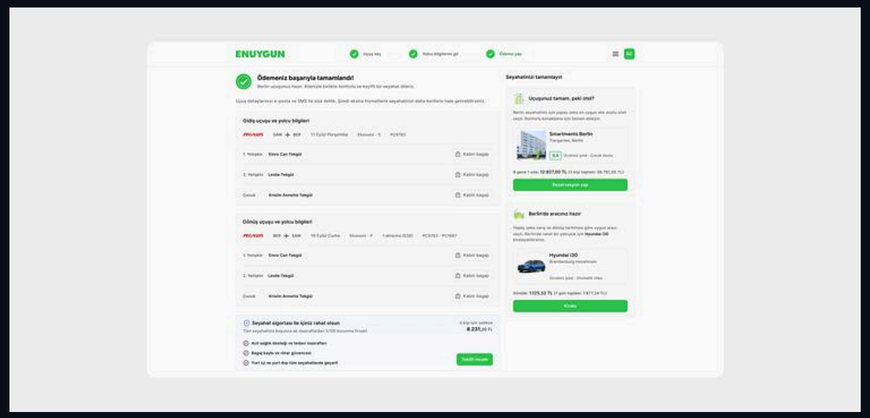


Cloud infrastructure

An instance moves through lifecycle states before it is ready to use.

What if a travel booking only partly succeeds?

A travel booking may involve several independent reservations and payments.



1 · Flight booked

Reservation confirmed.

2 · Hotel fails

No room is available.

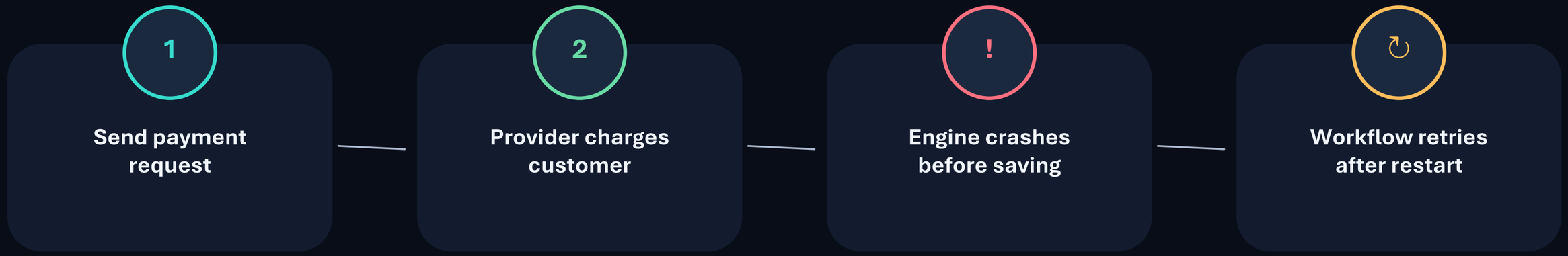
3 · Compensate

Cancel/refund the flight reservation where supported.

This is the motivation for Saga-style compensation: undo completed steps when a later step cannot proceed.

A crash can happen at exactly the wrong moment

Imagine a payment provider accepts the charge, but the workflow engine crashes before persisting the result.

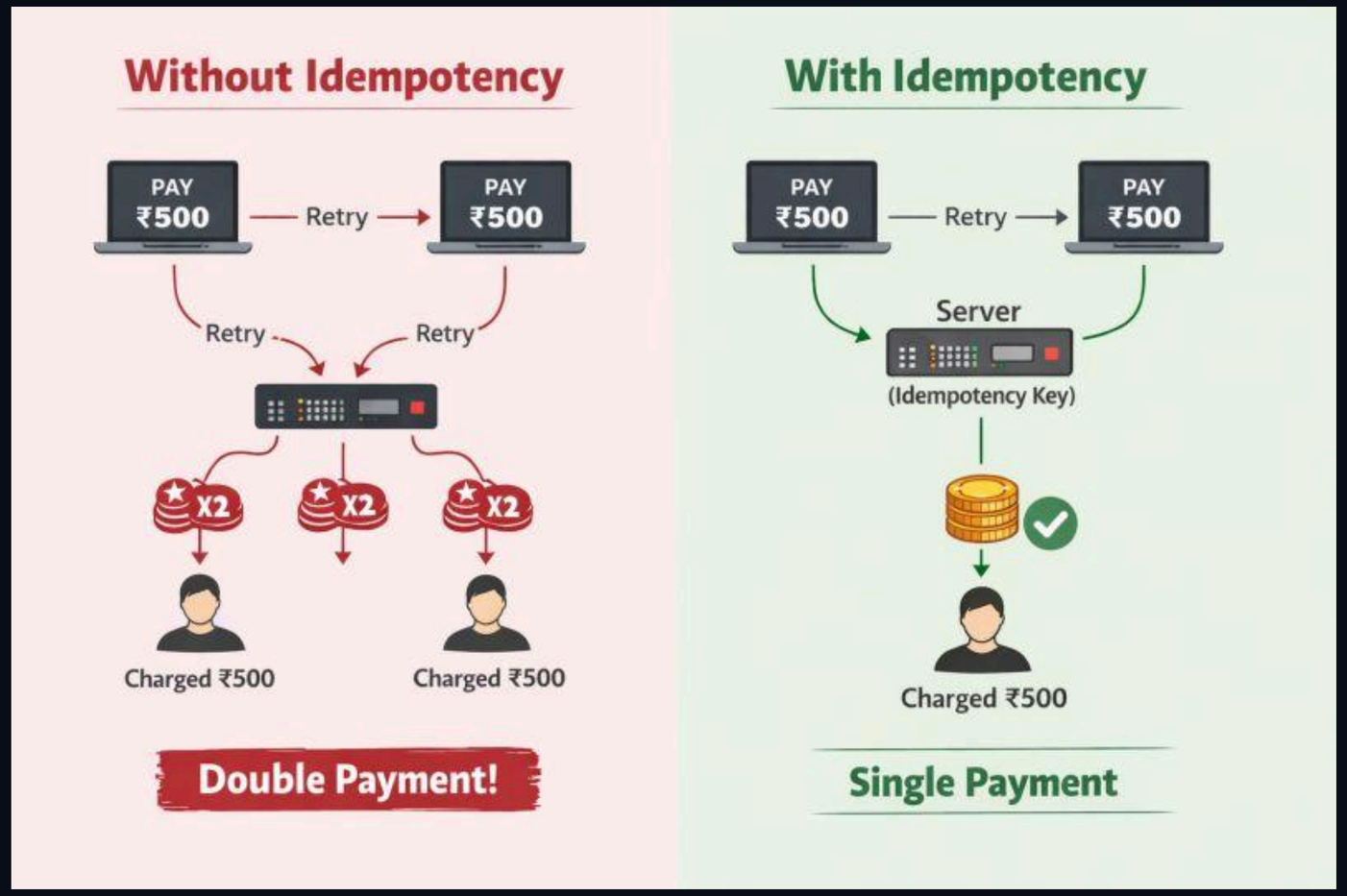


Without protection, the retry may charge the customer twice.

A durable engine must recover progress and make repeated attempts safe.

Same request key. One effective payment.

A stable idempotency key lets a payment service recognise a repeated request.



First attempt

Payment succeeds, but the response is lost.

Retry

The engine sends the same idempotency key.

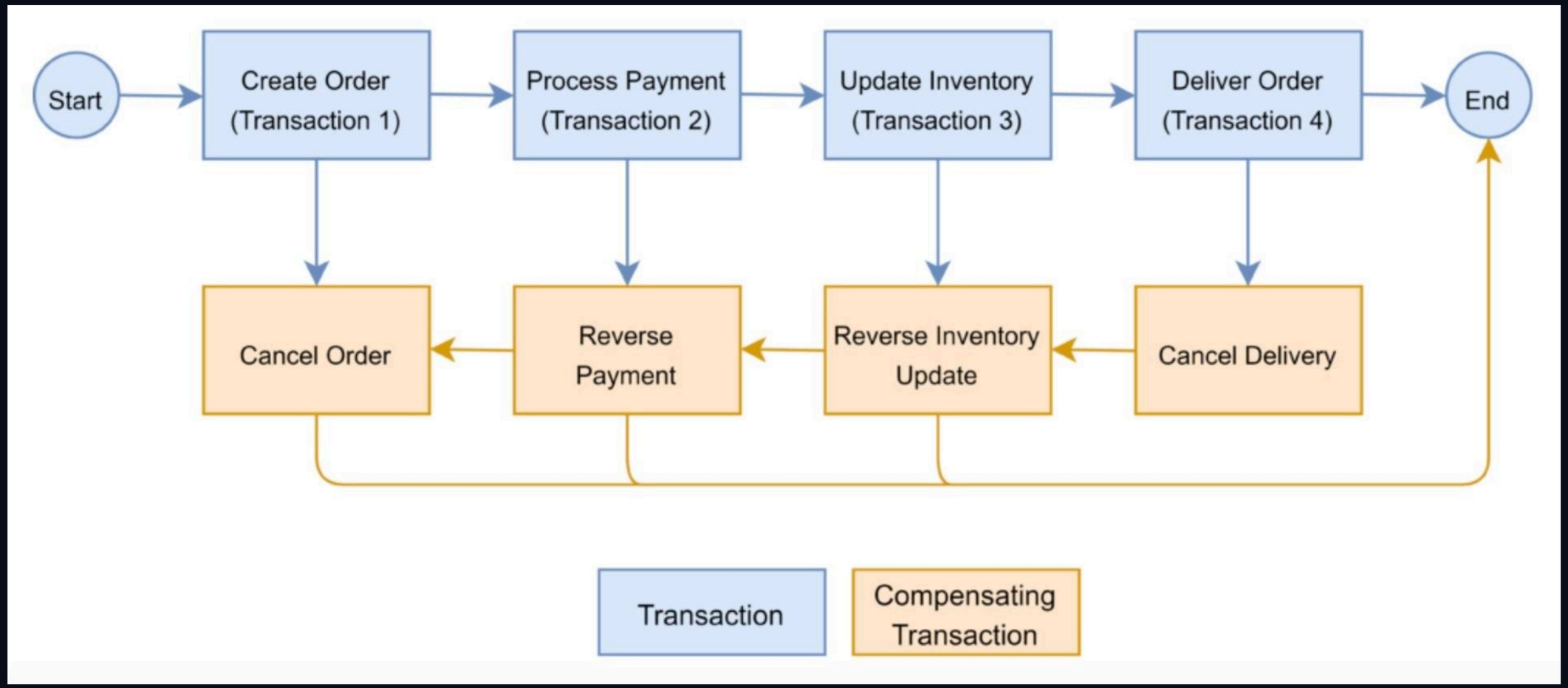
Safe outcome

Provider returns the original result.

Important: this depends on the downstream provider supporting and correctly enforcing idempotency.

If a later step fails, reverse earlier successful steps

A Saga uses compensating actions rather than assuming one global transaction can undo everything.



Example: cancel delivery → reverse inventory update → reverse payment → cancel order (where each compensation is possible).

A lightweight durable workflow execution engine

The project focuses on the core mechanisms needed to execute multi-step workflows reliably.

Workflow definition
Tasks + dependencies

Scheduler
Find eligible work

Workers
Execute handlers

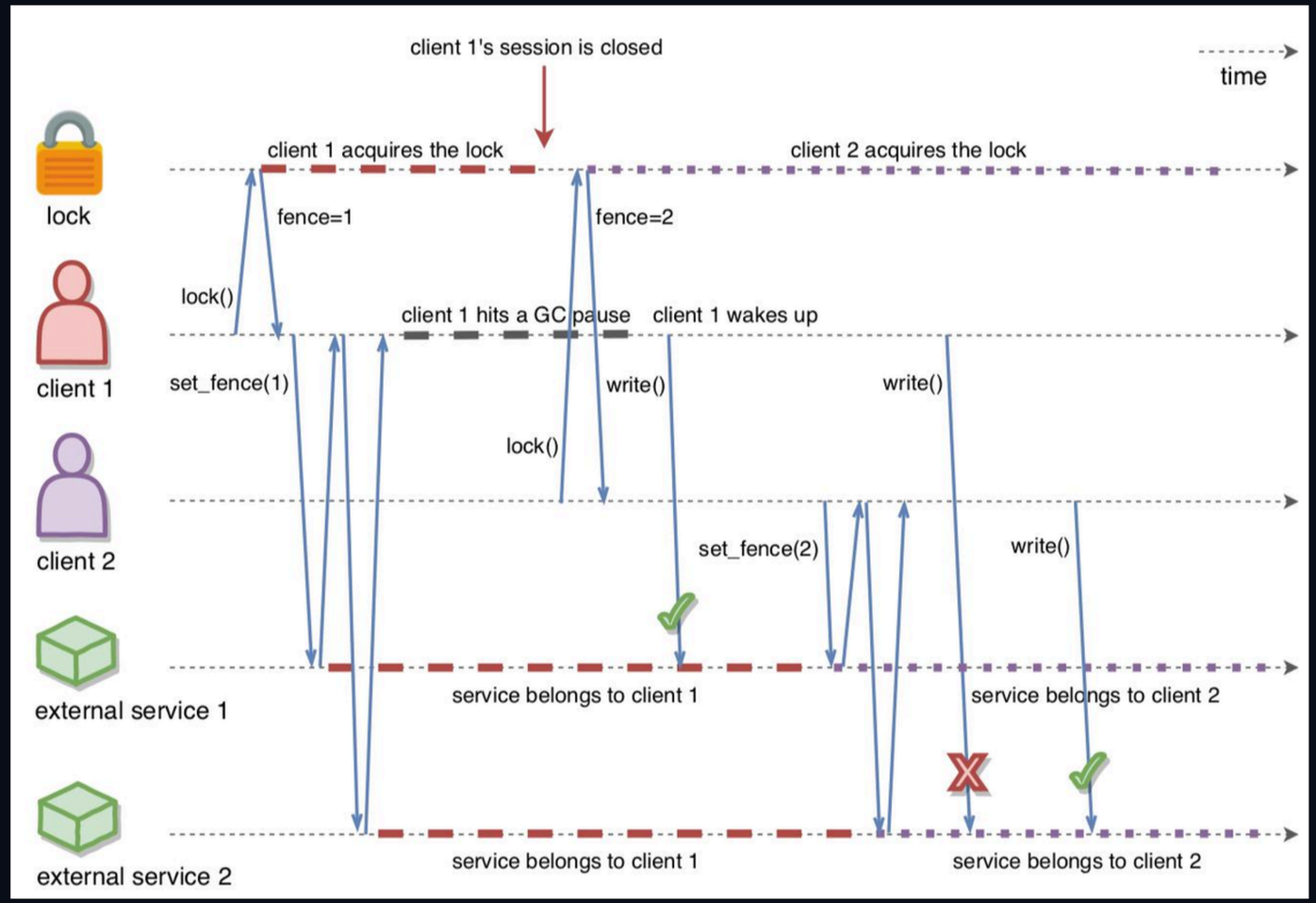
Persistent store
Runs + attempts + events

Recovery
Resume after restart

Persist state before advancing; on restart, reconstruct work from durable records, not worker memory.

What if two workers try to complete the same task?

Leases allow abandoned work to be reclaimed; fencing rejects stale workers.



Worker A

Claims the task; then pauses or crashes.

Worker B

Lease expires; B reclaims and completes it.

Fencing

A returns late; its stale write is rejected.

How do we know the design is correct?

We describe the system as a state machine, state properties, and use a model checker to explore possible behaviours.

1 · Specify

Represent workflow states, task states, workers, leases and transitions in TLA+.

2 · State properties

Safety: no invalid transitions, no stale completion.
Liveness: recoverable work can make progress under assumptions.

3 · Model-check

Use TLC to explore reachable states and inspect counterexample traces.

A checked model is not automatically a proof that the TypeScript implementation conforms to it.

A working prototype plus verification evidence

Keep the implementation focused; make the correctness claims precise and defensible.

Prototype

Workflow definitions
Task scheduling
Persistent state
Retries and recovery
Saga compensation

Formal verification

Shared state model
Safety + selected liveness properties
TLC checks
Counterexample analysis

Evaluation

Failure-injection demos
Implementation tests
Model/implementation comparison
Limitations and assumptions

Out of scope for the first version: replacing a production platform such as Temporal or guaranteeing exactly-once effects for arbitrary external services.

THE PROJECT IN ONE QUESTION

Can we make multi-step software workflows reliable, even when things fail?

Build a durable engine. Specify its behaviour. Explore failure scenarios. Check the properties that matter.

Questions & feedback